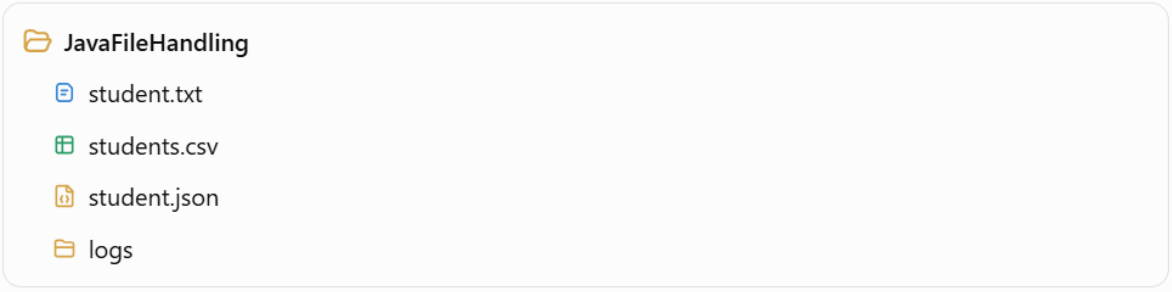
1. What is a file?

A file is a named collection of data stored permanently on a storage device.

For example, suppose you have this folder:



2. Java classes used for file handling

Java provides file-handling functionality mainly through the `java.io` and `java.nio.file` packages.

There are two important approaches:

Package	Purpose
`java.io`	Traditional file handling
`java.nio.file`	Modern file handling

Some important classes:

Class	Purpose
`File`	Represents a file or directory
`FileWriter`	Writes character data
`FileReader`	Reads character data
`BufferedWriter`	Efficiently writes text
`BufferedReader`	Efficiently reads text
`FileInputStream`	Reads binary data
`FileOutputStream`	Writes binary data
`Files`	Modern file operations
`Path`	Represents a file path

Program:

Java

```java
import java.io.File;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try {
            File file = new File("student.txt");

            if (file.createNewFile()) {
                System.out.println("File created successfully");
            } else {
                System.out.println("File already exists");
            }

        } catch (IOException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}
```

```
C:\Users\POORNA SAI\Desktop\JAVA>javac Main.java

C:\Users\POORNA SAI\Desktop\JAVA>java Main
File created successfully

C:\Users\POORNA SAI\Desktop\JAVA>java Main
File already exists

C:\Users\POORNA SAI\Desktop\JAVA>javac Main.java

C:\Users\POORNA SAI\Desktop\JAVA>java Main
File already exists

C:\Users\POORNA SAI\Desktop\JAVA>
```

Method	Explanation
`exists()`	Checks whether a file exists
`createNewFile()`	Creates a new file
`getName()`	Returns the file name
`getAbsolutePath()`	Returns the absolute path
`canRead()`	Checks read permission
`canWrite()`	Checks write permission
`isFile()`	Checks whether it is a file
`isDirectory()`	Checks whether it is a directory
`length()`	Returns file size in bytes
`delete()`	Deletes a file or empty directory
`mkdir()`	Creates one directory
`mkdirs()`	Creates directories including missing parent directories
`list()`	Returns names inside a directory

However with the help of File class alone we can't write or update the file , so we need another class **FileWriter** .

3. Writing content into a file

For writing, we need another class, such as `FileWriter`.

Java

```java
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;

public class Main {
    public static void main(String[] args) throws IOException {

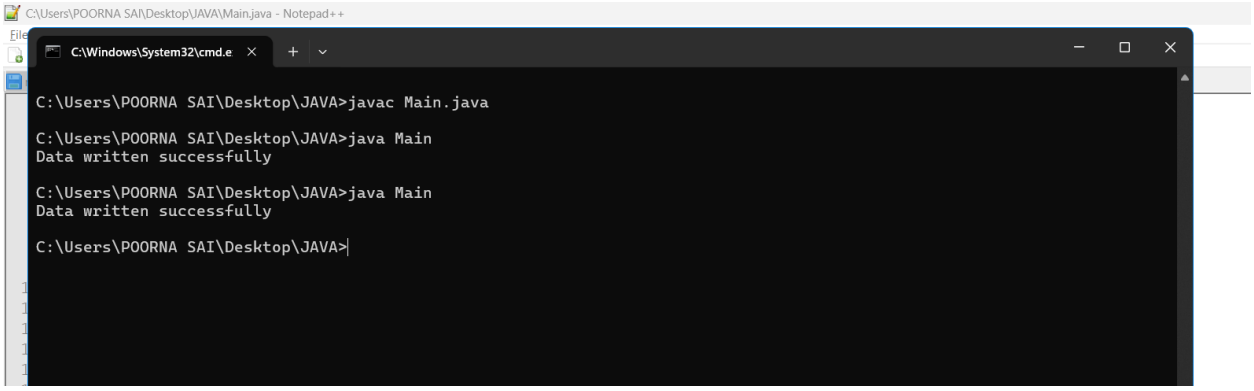
        File file = new File("student.txt");

        file.createNewFile();

        FileWriter writer = new FileWriter(file);

        writer.write("Hello Java!");
        writer.close();

        System.out.println("Data written successfully");
    }
}
```

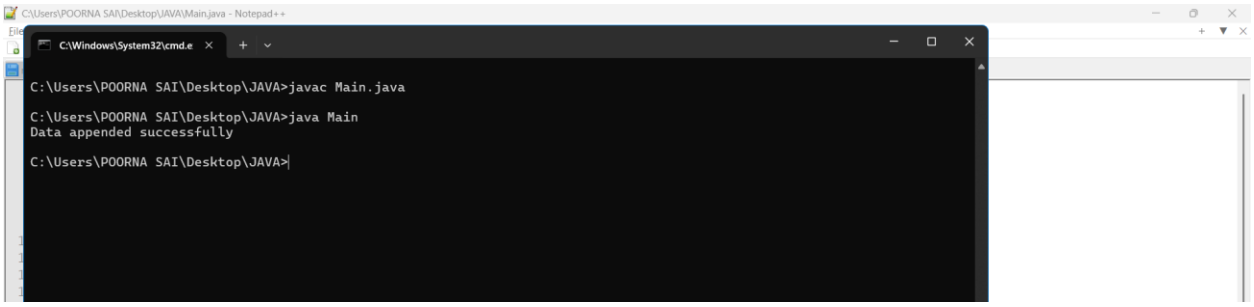


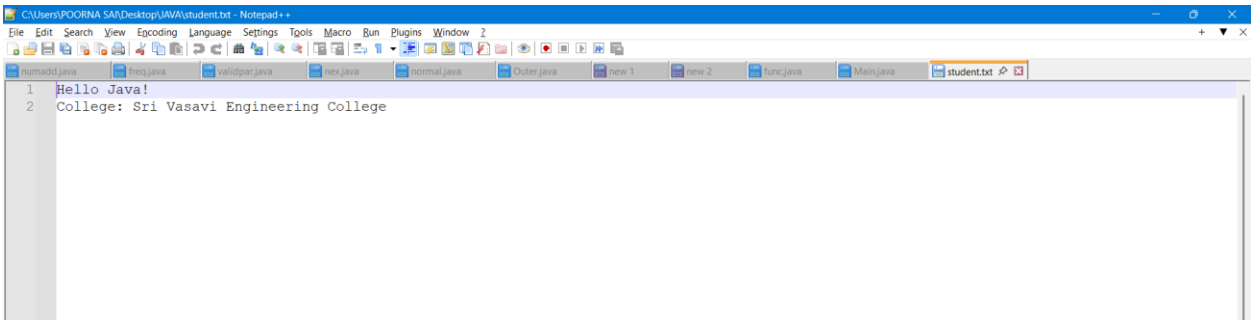
But it will override the content if you want to update the file by keeping the old data then we have to use another parameter :

FileWriter writer = new FileWriter("student.txt", true);

true → Adds content without removing existing content.

false → Overwrites existing content.





Using the **FileReader** for reading the content in the file.

```java
import java.io.FileReader;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try {
            FileReader reader = new FileReader("student.txt");

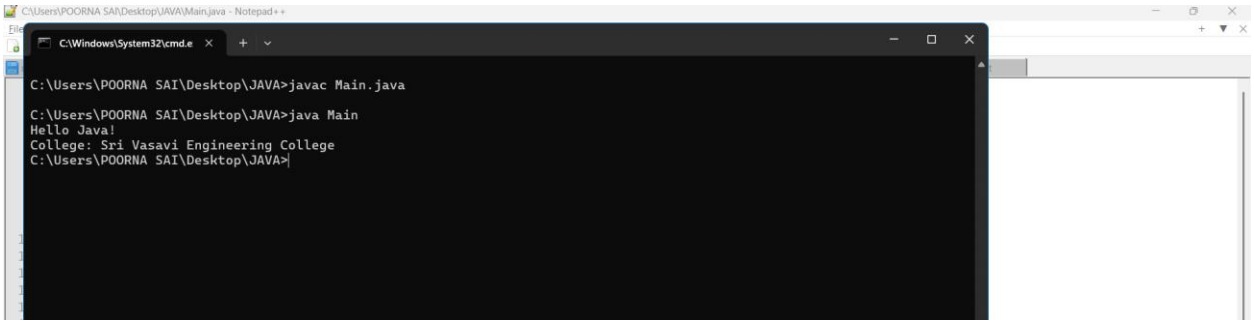
            int ch;

            while ((ch = reader.read()) != -1) {
                System.out.print((char) ch);
            }

            reader.close();

        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

OUTPUT :



2. BufferedWriter

BufferedWriter is a wrapper around another Writer (such as FileWriter).

It uses an internal memory buffer to temporarily store characters before writing them to the file.

```java
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {

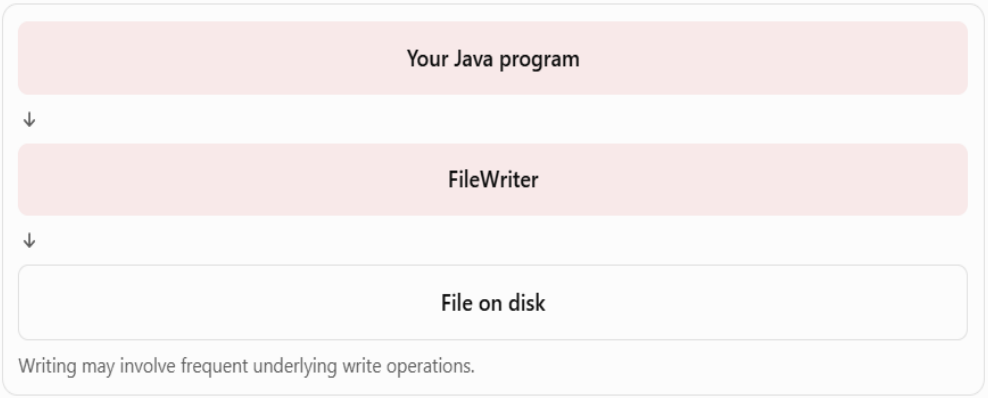
        FileWriter fw = new FileWriter("student.txt");

        BufferedWriter bw = new BufferedWriter(fw);

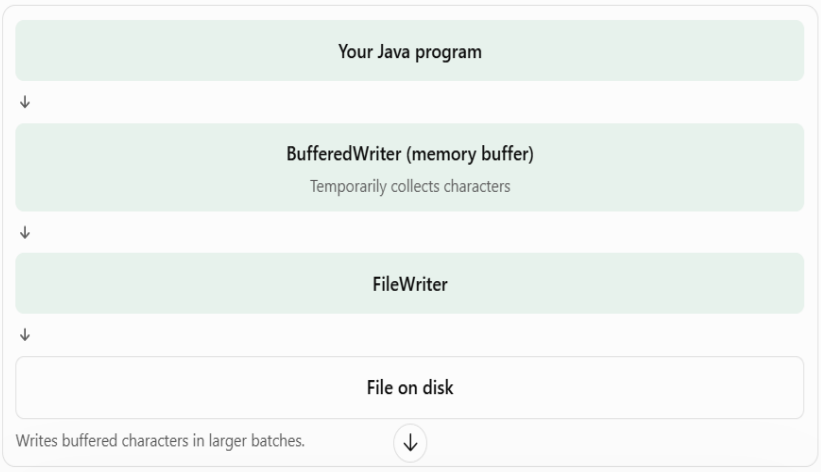
        bw.write("Hello Java");
        bw.newLine();
        bw.write("Welcome to Java");

        bw.close();
    }
}
```

Without BufferedWriter:



With BufferedWriter:



3. Difference between FileWriter and BufferedWriter

Feature	FileWriter	BufferedWriter
Purpose	Writes characters	Buffers characters before writing
Buffering	No additional character buffer	Yes
Performance	Can be less efficient for many small writes	Generally more efficient
`newLine()`	No	Yes
Can work independently?	Yes	No, needs another Writer

1. File class

- Package: `java.io`
- Introduced in: Java 1.0
- Used to represent file and directory paths.
- Provides methods to create, delete, rename and check files.

2. Files class

- Package: `java.nio.file`
- Introduced in: Java 7
- It is a utility class containing static methods.
- Provides operations to read, write, copy, move and delete files.

Java

```java
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;

public class Main {
    public static void main(String[] args) throws IOException {

        Path path = Path.of("student.txt");

        // Write data
        Files.writeString(path, "Hello Java");

        // Read data
        String data = Files.readString(path);

        System.out.println(data);
    }
}
```

Notice that we don't create an object of `Files`. We directly call its static methods.

3. Differences between File and Files

Feature	File	Files
Package	`java.io`	`java.nio.file`
Type	Regular class	Utility class
Introduced	Java 1.0	Java 7
Object creation	Yes	No
Methods	Instance methods	Static methods
Read content	Not directly	Yes
Write content	Not directly	Yes
Copy files	Not directly	Yes
Move files	Rename using `renameTo()`	Yes
Delete files	Yes	Yes
File information	Yes	Yes

Using File:

```java
File file = new File("student.txt");

if (file.delete()) {
    System.out.println("Deleted successfully");
}
```

Using Files:

```java
Path path = Path.of("student.txt");

Files.delete(path);
```

Both can delete a file, but `Files.delete()` throws an `IOException` if deletion fails, while `File.delete()` returns `false`.

Important: `writeString()` overwrites existing content by default.

To append content:

```java
Files.writeString(
    path,
    "\nWelcome to Java",
    StandardOpenOption.APPEND
);
```

4. Deletion methods

Method	Purpose
`Files.delete(path)`	Deletes a file or empty directory
`Files.deleteIfExists(path)`	Deletes if it exists; otherwise returns false

Paths.get() Vs Path.of() :-

1. Using `Paths.get()`

```java
import java.nio.file.Path;
import java.nio.file.Paths;

public class Main {
    public static void main(String[] args) {

        Path path = Paths.get("student.txt");

        System.out.println(path);
    }
}
```

- `Paths` is a utility class.
- `get()` is a static method.
- It returns a `Path` object.

`Paths.get()` has been available since Java 7.

2. Using `Path.of()`

```java
import java.nio.file.Path;

public class Main {
    public static void main(String[] args) {

        Path path = Path.of("student.txt");

        System.out.println(path);
    }
}
```

- `Path` is an interface.
- `of()` is a static factory method.
- It returns a `Path` object.

`Path.of()` was introduced in Java 11.

Important Differences : -

File vs Path

File	Path
Traditional API	Modern API
`java.io`	`java.nio.file`
Represents a file or directory	Represents a filesystem path
Has basic file operations	Works with the richer Files API

Difference:

`Files.list()`	`Files.walk()`
Lists immediate entries	Traverses nested directories
Non-recursive	Recursive by default
Returns Stream of Path	Returns Stream of Path

For large directory trees, be mindful of resource usage and traversal depth. `Files.walk(path, maxDepth)` lets you limit recursion.

FileWriter vs BufferedWriter

FileWriter	BufferedWriter
Writes characters	Writes buffered characters
Direct character writing	Reduces frequent I/O operations
Does not provide `newLine()`	Provides `newLine()`
Can be wrapped by BufferedWriter	Can wrap FileWriter

FileInputStream vs FileReader

FileInputStream	FileReader
Reads bytes	Reads characters
Binary files	Text files
No automatic character decoding	Performs character decoding
Extends InputStream	Extends InputStreamReader

`read()` vs `readLine()`

`read()`	`readLine()`
Reads one character	Reads one complete line
Returns an integer	Returns a String
Returns -1 at EOF	Returns null at EOF

`write()` vs `append()`

In `FileWriter`, append mode is controlled by the constructor: